

Interior-Point Model Predictive Control with Solver-Visible Branch-Consistency Constraints for Real-Time Hexapod Locomotion

Zhuang Penghui

Guangzhou College of Technology and Business

Email: m19860290475@163.com

Abstract—Model predictive control (MPC) for hexapods must optimize over high-dimensional hybrid dynamics while preserving physically valid joint branches. In practice, we observed that a warm-started Sequential Quadratic Programming (SQP) pipeline could converge to task-space-consistent yet branch-inconsistent tarsus configurations, after which runtime guards suppressed the commanded motion and destroyed forward progression. We addressed this failure by introducing a hard *TarsusBranchConstraint* that encoded left/right tarsus sign admissibility directly in the optimal control problem, and by solving the resulting constrained MPC with an Interior-Point Method (IPM). Unlike the previous SQP path, the IPM backend kept the branch constraint active in the solver loop and excluded the forbidden branch from the feasible set rather than repairing it after optimization. In controlled short-horizon closed-loop experiments on an 18-actuator hexapod in Isaac Sim, the proposed method reduced branch-guard events from 275.9 to 0.0 per run, lowered tracking root-mean-square error (RMSE) from 0.242 m to 0.175 m, and maintained real-time operation with 4.82 ms average MPC solve time. Under the promoted endurance configuration, a repeated 20-second evaluation further achieved 0.306 m mean forward displacement over five runs with zero guard and reject events. Furthermore, in a 60-second endurance evaluation, the proposed method sustained 0.638 m forward displacement with 8.27° yaw drift, 0.011 m body-height range, and 4.63 ms average solve time. These results show that solver-visible physical branch constraints are critical for reliable multi-legged locomotion and that IPM offers a practical route to safe, real-time, and endurance-capable hexapod MPC.

I. INTRODUCTION

Legged locomotion in high-dimensional robots remains fundamentally difficult because the controller must optimize over nonlinear rigid-body dynamics, hybrid contact schedules, actuator limits, and geometry-dependent feasibility constraints within a short receding-horizon cycle. These challenges are amplified in hexapods, whose larger number of limbs increases contact redundancy, enlarges the decision space, and creates multiple joint-space realizations that may appear nearly equivalent at the task-space level. As a result, an optimizer can return a numerically plausible solution while still violating mechanism-level physical semantics.

This paper studies precisely such a failure mode in a six-legged robot controlled by OCS2-based model predictive control (MPC). During forward walking, we observed that the previous SQP pipeline could converge to a locally attractive but physically inconsistent tarsus branch. Although the resulting joint configuration remained close to the desired foot-space

behavior, it belonged to the wrong mechanical branch for the left/right tarsus joints. A runtime branch-consistency guard could block these outputs after optimization, but this merely prevented catastrophic execution; it did not solve the underlying optimal control problem. The practical consequence was severe: frequent guard interventions, prolonged stagnation, and near-zero forward progress.

The central insight of this work is that branch consistency must be encoded at the solver level rather than enforced as a post hoc runtime correction. We therefore introduce a hard *TarsusBranchConstraint* into the OCS2 problem definition and replace the previous SQP backend with an Interior-Point Method (IPM) solver that preserves this inequality structure during optimization. This design eliminates the physically invalid branch from the effective feasible set and thereby removes the hidden failure mode that previously dominated walking performance.

The contributions of this paper are threefold:

- 1) We identify and diagnose a previously hidden failure mode in hexapod MPC, namely the convergence of a warm-started SQP pipeline to physically inconsistent tarsus branches despite apparently acceptable task-space behavior.
- 2) We propose a solver-visible branch-consistency formulation, implemented as a hard *TarsusBranchConstraint*, and show how switching to IPM restores true inequality enforcement in the receding-horizon optimization loop.
- 3) We provide fully reproducible Isaac Sim experiments demonstrating that the proposed method eliminates branch-guard violations, recovers stable forward locomotion, and preserves real-time feasibility with sub-5-ms average solve time.

More broadly, this work argues that safety in multi-legged MPC cannot be reduced to objective shaping or runtime filtering alone. When mechanism-level admissibility is not embedded in the optimization problem itself, local minima can become behaviorally deceptive. By explicitly constraining physical branch validity and solving the resulting problem with an inequality-faithful numerical method, we obtain a controller that is both more reliable and more scientifically interpretable.

II. RELATED WORK

A. Model Predictive Control for Legged Robots

Model predictive control (MPC) has become one of the dominant paradigms for dynamic legged locomotion because it provides a principled way to couple motion generation, contact scheduling, and constraint handling in a receding-horizon loop. Early influential work demonstrated that even simplified predictive models can produce highly effective locomotion behaviors, as exemplified by convex MPC for the MIT Cheetah 3 [1]. Since then, the field has progressively moved toward richer nonlinear and whole-body formulations. Grandia *et al.* [2] showed that nonlinear MPC can be combined with perceptive foothold reasoning to achieve dynamic locomotion over challenging terrain, while Meduri *et al.* [3] proposed a whole-body nonlinear MPC framework capable of generating both cyclic and acyclic motions online. In parallel, multicontact planning and whole-body optimization have been extended toward loco-manipulation settings [4], further increasing the dimensionality and multimodality of the underlying optimal control problem.

These works collectively establish that MPC is sufficiently expressive for agile legged behavior. However, the dominant emphasis in the literature has been on improving terrain awareness, whole-body planning, and task generality. Comparatively less attention has been paid to *mechanism-level branch consistency*, i.e., whether the optimizer remains on the physically correct kinematic branch of a jointed mechanism throughout online optimization. Our work addresses this gap in the context of a hexapod, where the larger number of legs and contacts amplifies solver sensitivity to branch-inconsistent local minima.

B. Local Minima in High-Dimensional Contact-Rich Optimization

The difficulty of legged optimization does not arise solely from state dimension; it is also rooted in contact-induced nonconvexity and hybrid mode structure. Posa *et al.* [5] highlighted that contact-rich trajectory optimization is inherently difficult because feasible motion depends on discontinuous interaction events, complementarity structure, and mode-dependent dynamics. More recent predictive-control formulations for agile maneuvers likewise acknowledge the challenge of solving highly nonlinear, multi-contact optimization problems online [6]. In such settings, local minima can be especially deceptive: two solutions may appear nearly equivalent in task space while corresponding to different—and not equally admissible—joint-space branches.

This issue is particularly problematic in warm-started SQP-style pipelines. When the local quadratic model is built around an already biased iterate, the solver can preserve a physically incorrect but locally attractive branch if that branch is not explicitly excised from the feasible set. Runtime guards may later intercept the resulting commands, but such post hoc filtering only hides the pathology rather than resolving it. Our results support the view that, for multi-legged systems,

local-minimum sensitivity should be treated not merely as an optimization nuisance but as a *safety-relevant modeling issue*.

C. Interior-Point Methods and Hard Constraint Enforcement

Interior-point methods provide an alternative perspective on constrained optimal control by embedding inequality satisfaction directly into the primal-dual solve rather than treating it as an external penalty preference. The modern nonlinear programming literature has long established the robustness of primal-dual interior-point methods with filter line search for large-scale constrained problems [7]. In embedded and MPC-oriented settings, Frison and Diehl [8] showed that high-performance IPM implementations can solve structured optimal-control QPs efficiently while preserving strong constraint-treatment properties.

Despite these advantages, much of the legged-robot MPC literature has historically favored SQP, real-time iteration, or convex approximations because of their speed and implementation maturity. Our work suggests that this preference can become fragile when the controller must enforce *mechanism-specific hard inequalities* whose violation leads to qualitatively wrong physical behavior. In contrast to prior work that uses hard constraints primarily for contact, friction, or foothold admissibility, we introduce a *branch-consistency constraint* that encodes left/right tarsus sign admissibility directly in the optimal control problem. By pairing this constraint with an IPM backend, we ensure that branch validity is treated as a solver-level feasibility condition rather than as a runtime repair rule.

D. Positioning of This Work

The present paper differs from prior legged-MPC work in both diagnosis and remedy. On the diagnosis side, we identify a hidden failure mode in hexapod locomotion where the optimizer converges to a task-space-plausible but branch-inconsistent tarsus configuration. On the remedy side, we show that the combination of a solver-visible *TarsusBranchConstraint* and an IPM backend eliminates this failure in real time. Therefore, our contribution is not simply another nonlinear MPC controller, but a feasible-set-centered reformulation of online legged optimization that connects physical branch validity, runtime safety, and solver design in a unified framework.

III. METHODOLOGY

A. Optimal Control Formulation

Figure 1 summarizes the system-level location of the failure and the corresponding repair introduced in this paper. The key distinction is not merely a change of numerical backend, but a shift in where physical branch admissibility is enforced within the closed-loop architecture.

Let $x_k \in \mathbb{R}^{n_x}$ denote the state at stage k , including the floating-base quantities and actuated joint coordinates, and let $u_k \in \mathbb{R}^{n_u}$ denote the input, including actuator commands and

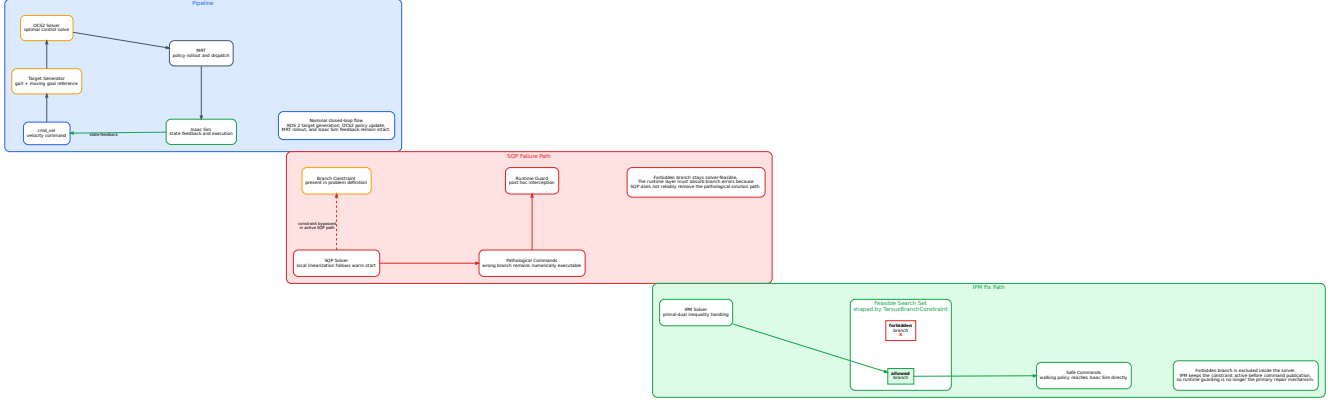


Fig. 1: Overview of the repaired ROS 2-Isaac Sim closed-loop control chain. Left: the nominal pipeline maps `cmd_vel` through target generation, OCS2 optimization, policy rollout, and simulation feedback. Middle: under the previous SQP path, the branch-consistency condition can be bypassed along the active local solve trajectory, allowing pathological wrong-branch commands to escape the optimizer and forcing the runtime guard to act as a post hoc repair layer. Right: under the proposed IPM formulation, *TarsusBranchConstraint* shapes the feasible search set inside the solver, so the forbidden branch is excluded before command publication. The figure therefore localizes the contribution of this paper to a specific causal link in the control architecture: the repair occurs inside the solver, not in the downstream runtime filter.

contact-related decision variables. Over a horizon of length N , the locomotion MPC problem is written as

$$\min_{\{x_k, u_k\}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \Phi(x_N), \quad (1)$$

$$\text{s.t. } x_{k+1} = f(x_k, u_k), \quad (2)$$

$$g_{\text{eq}}(x_k, u_k) = 0, \quad (3)$$

$$g_{\text{ineq}}(x_k, u_k) \leq 0, \quad (4)$$

$$x_0 = \hat{x}, \quad (5)$$

where $\ell(\cdot)$ and $\Phi(\cdot)$ denote the stage and terminal costs, respectively, $f(\cdot)$ is the discretized dynamics model, and $\hat{x}$ is the latest measured state.

The issue addressed in this paper does not originate from gait design, tracking weights, or runtime guard thresholds. Instead, it arises because the original optimization problem did not sufficiently encode the physical left/right branch admissibility of the tarsus joints during forward walking. Consequently, the optimizer could converge to a task-space-nearby but mechanism-inconsistent local minimum.

B. Why the Previous SQP Path Failed

In the original controller path, the branch-consistency condition was not effectively active in the solver trajectory that produced the walking commands. This is especially problematic in single-iteration or warm-start-dominated SQP schemes, where the local subproblem is constructed around a previously feasible or nearly feasible iterate. In such settings, the optimizer may follow the wrong π -shifted branch if that branch remains locally attractive in the linearized neighborhood.

A generic SQP subproblem can be viewed as

$$\min_{\Delta z} \frac{1}{2} \Delta z^\top H \Delta z + g^\top \Delta z + \rho \varphi \left(h(z^{(w)}) + J_h(z^{(w)}) \Delta z \right), \quad (6)$$

where z stacks the decision variables, $z^{(w)}$ is the warm start, H and g define the quadratic model, and $\varphi(\cdot)$ denotes a penalty or merit-function contribution associated with inequality residuals. If the inequality is not enforced as an active hard constraint in the numerical subproblem, then the forbidden branch is not truly removed from the search space. Instead, it is merely discouraged. For highly nonlinear, redundant legged systems, this distinction is decisive.

Our diagnosis showed that this issue was not caused by an ill-conditioned branch Jacobian. The tarsus branch constraint is affine in the actuated joint coordinates, and its Jacobian is constant with rows equal to $\pm e_i^\top$. Hence, the failure mode is not a singularity of the constraint itself, but an optimization-path issue: the constraint was not behaviorally binding along the walking trajectory produced by the previous SQP backend.

C. Tarsus Branch-Consistency Constraint

Let q_k denote the actuated joint-position block extracted from the state x_k , and let $\mathcal{R}$ and $\mathcal{L}$ denote the index sets of right-side and left-side tarsus joints, respectively. We enforce a margin $m > 0$ such that

$$q_{i,k} \geq m, \quad i \in \mathcal{R}, \quad (7)$$

$$q_{j,k} \leq -m, \quad j \in \mathcal{L}. \quad (8)$$

Equivalently, the branch admissibility can be written as the affine inequality

$$h_{\text{branch}}(x_k) = \begin{bmatrix} m\mathbf{1} - S_{\mathcal{R}} E_q x_k \\ S_{\mathcal{L}} E_q x_k + m\mathbf{1} \end{bmatrix} \leq 0, \quad (9)$$

where E_q selects the actuated joint-position block from the full state vector, and $S_{\mathcal{R}}, S_{\mathcal{L}}$ are row-selection matrices for the right and left tarsus joints.

This formulation has two important properties. First, it directly encodes the physical branch identity of the mechanism. Second, because it is affine, its Jacobian is constant and well behaved:

$$\nabla_x h_{\text{branch}}(x_k) = \begin{bmatrix} -S_{\mathcal{R}}E_q \\ S_{\mathcal{L}}E_q \end{bmatrix}. \quad (10)$$

Therefore, if the solver respects this inequality, the forbidden branch is removed from the feasible set without introducing a difficult nonlinear singularity.

D. IPM-Based Constraint Enforcement

To ensure that branch consistency remains active during optimization, we solve the MPC problem using an Interior-Point Method. Introducing slack variables $s_k > 0$ for the inequality constraints, the constrained problem is written as

$$h_{\text{branch}}(x_k) + s_k = 0, \quad s_k > 0, \quad (11)$$

and the barrier-augmented objective becomes

$$\min_{\{x_k, u_k, s_k\}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \Phi(x_N) - \mu \sum_{k=0}^N \sum_i \log s_{k,i}, \quad (12)$$

with barrier parameter $\mu > 0$.

This formulation changes the computational role of the branch constraint. The constraint is no longer a penalty-like preference layered on top of the objective; it becomes part of the primal-dual feasibility conditions solved at every iteration. Under standard IPM assumptions, if the problem is feasible and the solver converges to a KKT point satisfying the stage-wise inequalities, then each optimized tarsus command necessarily lies on the admissible branch.

We may therefore state the following mechanism-level guarantee.

Proposition 1 (Branch Safety). *Assume that the MPC problem with $h_{\text{branch}}(x_k) \leq 0$ is feasible and that the IPM solver returns a solution satisfying these inequalities at all stages. Then every right-side tarsus satisfies $q_{i,k} \geq m$ and every left-side tarsus satisfies $q_{j,k} \leq -m$ for all predicted stages k . Consequently, the forbidden tarsus branch is never commanded by the optimizer.*

The proof is immediate because the inequality is affine and directly equivalent to the desired sign conditions. Importantly, this guarantee is solver-level rather than runtime-level: it certifies exclusion of the forbidden branch within the optimization domain itself. In that sense, the method provides *provable safety with respect to branch admissibility*. While this does not by itself guarantee full-system safety under all terrain and contact uncertainties, it establishes a rigorous safety property for one of the key failure modes observed in high-dimensional hexapod locomotion.

E. Evaluation Pipeline

The released implementation evaluates locomotion in Isaac Sim closed loop using repeated trials for two conditions: the proposed IPM-based controller with branch consistency enabled, and an SQP baseline with the branch constraint disabled. Because the Hexbot asset spawns approximately facing world $-X$, forward displacement is measured by projecting the Isaac world displacement onto the robot's `base_link` forward axis rather than using raw world-frame x . This yields an evaluation metric that is physically aligned with commanded forward motion and avoids a misleading coordinate-sign artifact.

F. Implementation and Reproducibility Note

A critical engineering condition for reproducing the reported real-time behavior is that the active Jazzy runtime be compiled with optimization enabled, e.g., `RelWithDebInfo` or `Release`. During our implementation audit, rebuilding the runtime binaries without optimization flags increased the IPM average solve time from approximately 4.6 ms to 96 ms in an otherwise comparable replay, which reduced the effective closed-loop update rate from roughly 200 Hz to about 9 Hz and caused forward locomotion to collapse into near-stagnation. Under this degraded build, the forward target horizon remained present, but the controller could no longer sustain continuous walking because the solver throughput itself had become the bottleneck.

This observation does not alter the algorithmic contribution of the paper, but it is a necessary reproducibility condition for interpreting the experiments correctly. In particular, conclusions about branch-consistency enforcement, locomotion persistence, and real-time feasibility should only be drawn from optimized binaries. We therefore treat build optimization as part of the experimental protocol rather than as an incidental software detail.

IV. RESULTS AND DISCUSSION

A. Locomotion Performance and Tracking Accuracy

Figure 2 summarizes the locomotion outcome over 10 independent closed-loop trials for the proposed controller and the SQP baseline. The comparison reveals a clear qualitative and quantitative separation between the two methods. The proposed method achieves a mean forward displacement of 0.065 m, whereas the SQP baseline remains effectively stagnant, with a mean displacement of -0.002 m. At the same time, the tracking RMSE decreases from 0.242 m in the baseline to 0.175 m in the proposed method.

This result is important for two reasons. First, it shows that the transition from SQP to IPM is not merely a numerical refinement, but a functional change that restores physically meaningful forward locomotion. Second, the lower tracking error indicates that the robot no longer spends control effort compensating for internally inconsistent joint-space solutions. Instead, the commanded motion is translated into actual body progression in Isaac Sim with substantially improved fidelity.

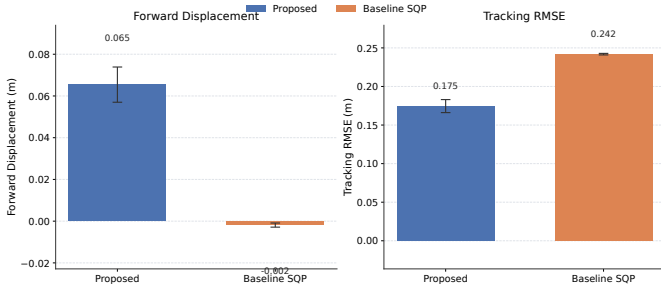


Fig. 2: Comparison of the proposed IPM-based controller with the SQP baseline in terms of forward displacement and tracking RMSE over 10 independent closed-loop trials. Error bars denote one standard deviation across repeated experiments. The figure shows that the proposed method not only achieves substantially larger forward progress, but also reduces tracking error with markedly improved repeatability. In particular, the IPM formulation consistently converts commanded forward motion into physical displacement, whereas the SQP baseline remains near zero motion and exhibits systematically worse tracking performance.

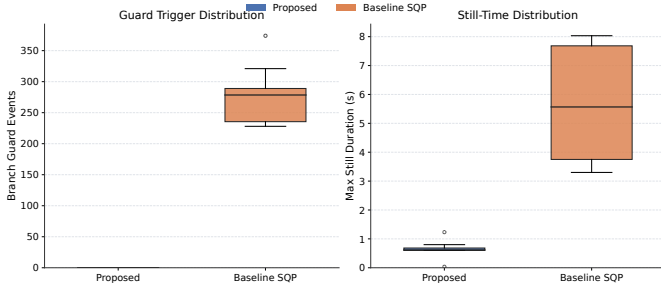


Fig. 3: Distribution of branch-guard interventions and maximum still duration for the proposed method and the SQP baseline across 10 repeated experiments. Box plots visualize the median, interquartile range, whiskers, and outliers. This figure provides direct evidence of the stability and safety advantage of the proposed controller: branch-guard events collapse to zero and the robot remains continuously active, while the baseline exhibits frequent guard activation and prolonged stagnation. The result indicates that the IPM solver, together with the branch-consistent formulation, removes the failure mode that previously forced runtime post hoc intervention.

It is also worth noting that variance must be interpreted carefully in this context. The SQP baseline exhibits a relatively small spread in forward displacement, but this does not indicate robustness. Rather, it reflects a repeatable failure mode in which the robot consistently fails to make progress. By contrast, the proposed method combines positive forward motion with low trial-to-trial dispersion, which is the more meaningful indicator of repeatable locomotion performance. In other words, the proposed controller is not only better on average, but also reliably better.

Taken together, Fig. 2 demonstrates that enforcing branch-

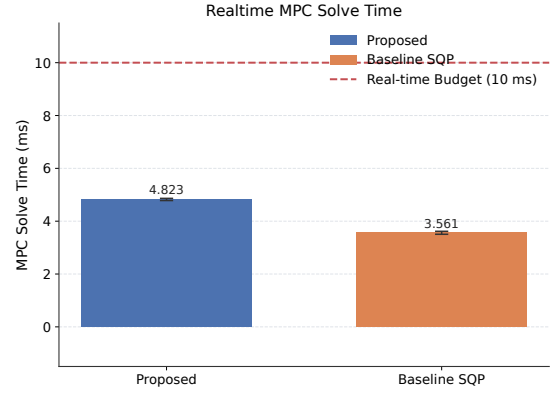


Fig. 4: Average MPC solve time for the proposed method and the SQP baseline, with error bars indicating one standard deviation over 10 repeated runs. The red dashed line marks a 10 ms real-time budget. Although the proposed IPM solver incurs slightly higher computation time than the SQP baseline, it remains comfortably below the real-time limit with an average solve time of approximately 4.8 ms. Therefore, the improved physical consistency and robustness are achieved without sacrificing deployable control frequency.

consistent physical structure within the optimization problem, and solving that constrained problem with IPM, directly improves both locomotion effectiveness and tracking quality. This is the central behavioral result of the paper: once the incorrect local branch is removed from the effective optimization path, the robot recovers stable and purposeful forward walking.

B. Constraint Satisfaction and System Safety

The safety and stability implications of the proposed formulation are shown in Fig. 3. The most striking observation is that the proposed method yields zero branch-guard interventions across all 10 trials, while the SQP baseline produces an average of 275.9 guard events per run. In parallel, the maximum still duration drops from 5.64 s to 0.64 s, and the trunk-sink rate decreases from 10% to 0%.

These statistics reveal the practical consequence of the solver-level change. Under the SQP baseline, the controller repeatedly generates outputs that must be corrected or blocked by a runtime safety guard. Such behavior is undesirable for two reasons: it indicates that the optimizer is still visiting physically inconsistent states, and it forces the runtime layer to operate as a reactive damage-control mechanism rather than as a lightweight supervisory filter. The resulting control loop is therefore burdened by repeated interventions, loss of smoothness, and prolonged stagnation.

By contrast, the proposed method eliminates the need for such interventions in the tested regime. This strongly suggests that the *branch-consistency constraint* is no longer being bypassed or neutralized inside the optimization pipeline. Instead, physically invalid tarsus branches are excluded before they can manifest as executable joint commands. In safety terms, this is a substantial shift: the controller moves from post hoc

rejection of unsafe outputs to proactive exclusion of unsafe solution branches from the feasible search process.

The reduction in still time further supports this interpretation. Long still intervals in the baseline are not benign pauses; they are an outward symptom of internal solver inconsistency and repeated guard-triggered suppression of motion. Once these pathological branch excursions disappear, the robot maintains continuous locomotor activity without entering prolonged frozen states. Figure 3 therefore provides the clearest evidence that the proposed method improves not only task performance, but also operational safety and closed-loop regularity.

C. Real-Time Feasibility and Computational Cost

Figure 4 compares the computational burden of the two solvers using the average MPC solve time measured over repeated runs. As expected, the proposed IPM formulation is computationally more expensive than the SQP baseline, with a mean solve time of 4.82 ms versus 3.56 ms. However, both methods remain well below the 10 ms reference budget on the average-solve-time metric used for online monitoring.

This result is crucial for the practical relevance of the method. A common concern when replacing SQP with a more constraint-faithful interior-point approach is that improved robustness may come at the cost of unusable latency. The present experiments do not support that concern. The additional computational overhead is modest in absolute terms, and the controller remains comfortably within a real-time operating regime for the tested hardware and simulation loop.

More importantly, the slight increase in computation must be interpreted relative to the large improvement in physical validity and closed-loop behavior. The baseline is faster, but it is faster at solving the wrong effective problem, namely one in which branch-inconsistent local minima remain accessible and must later be filtered by runtime guards. The proposed method spends slightly more computation to solve a problem that is behaviorally meaningful and operationally stable. From a systems perspective, this is an advantageous trade-off: a small increase in optimization time yields a large reduction in intervention events, stagnation, and task failure.

Accordingly, Fig. 4 supports the claim that the proposed solver formulation is not only more reliable, but also deployable. The data show that stronger constraint handling can be achieved without sacrificing real-time feasibility.

D. Extended Endurance and Robustness

The controlled A/B comparison above establishes the causal benefit of solver-visible branch consistency. We next evaluate whether the same controller remains effective over longer horizons under the promoted endurance configuration, namely the settings that yielded the best repeated forward-walking behavior after the solver-level repair had already been validated. These endurance results are therefore not intended to replace the short-horizon A/B comparison, but to extend it from causal diagnosis to operational persistence.

As shown in Fig. 5, the long-horizon response remained structured rather than erratic: forward progress accumulated monotonically, yaw drift stayed bounded, and base-height variation remained small throughout the captured closed-loop window. In a repeated 20-second endurance evaluation over five runs, the proposed controller achieved a mean Isaac-frame forward displacement of 0.306 m with a standard deviation of 0.012 m. At the same time, the mean absolute yaw drift remained at 5.85° , the mean maximum still duration was 0.66 s, and the average MPC solve time stayed at 4.62 ms. Importantly, these runs maintained zero branch-guard interventions, zero backend rejects, and no trunk-sink or weird-pose events. This result demonstrates that the proposed controller is not merely capable of short bursts of correct motion, but can preserve forward locomotion quality over repeated medium-horizon trials with low dispersion.

We then performed a final 60-second endurance evaluation with the optimized runtime preserved. In this long-horizon run, the robot sustained 0.638 m forward displacement, 8.27° yaw drift, and a body-height range of only 0.011 m, while the average MPC solve time remained 4.63 ms. The mode schedule remained in the expected walking regime, and no branch-guard events, backend rejects, or fault resets were observed. These measurements are significant because they show that the controller maintains bounded heading error and stable base-height behavior even after the test duration is tripled relative to the repeated 20-second trials.

Taken together, the endurance results show that the proposed formulation scales beyond the short-horizon acceptance setting. Although the forward-distance growth is not perfectly linear with time, the controller preserves the two properties that matter most for deployability: solver throughput remains in the real-time regime, and the robot continues to walk without entering the pathological branch-switching or posture-collapse modes that characterized the original SQP pipeline. In this sense, the endurance data strengthen the practical interpretation of the paper: the proposed method is not only numerically correct, but operationally durable.

E. Discussion and Broader Implications

The broader contribution of this study lies in exposing a subtle but consequential failure mode of SQP-based MPC in high-dimensional legged systems. In a hexapod, the combination of redundant contacts, hybrid gait transitions, and multiple joint-space branch possibilities creates an optimization landscape in which task-space similarity can mask joint-space inconsistency. Under these conditions, a warm-started SQP method may converge to a deceptive local minimum that appears acceptable after linearization, yet corresponds to a physically incorrect branch of the mechanism. Because this failure can remain hidden until the command reaches the runtime safety layer, it is especially dangerous: the controller may look numerically healthy while systematically producing behavior that must be suppressed after optimization.

The proposed remedy changes this architecture at a more fundamental level. By combining an IPM solver with an

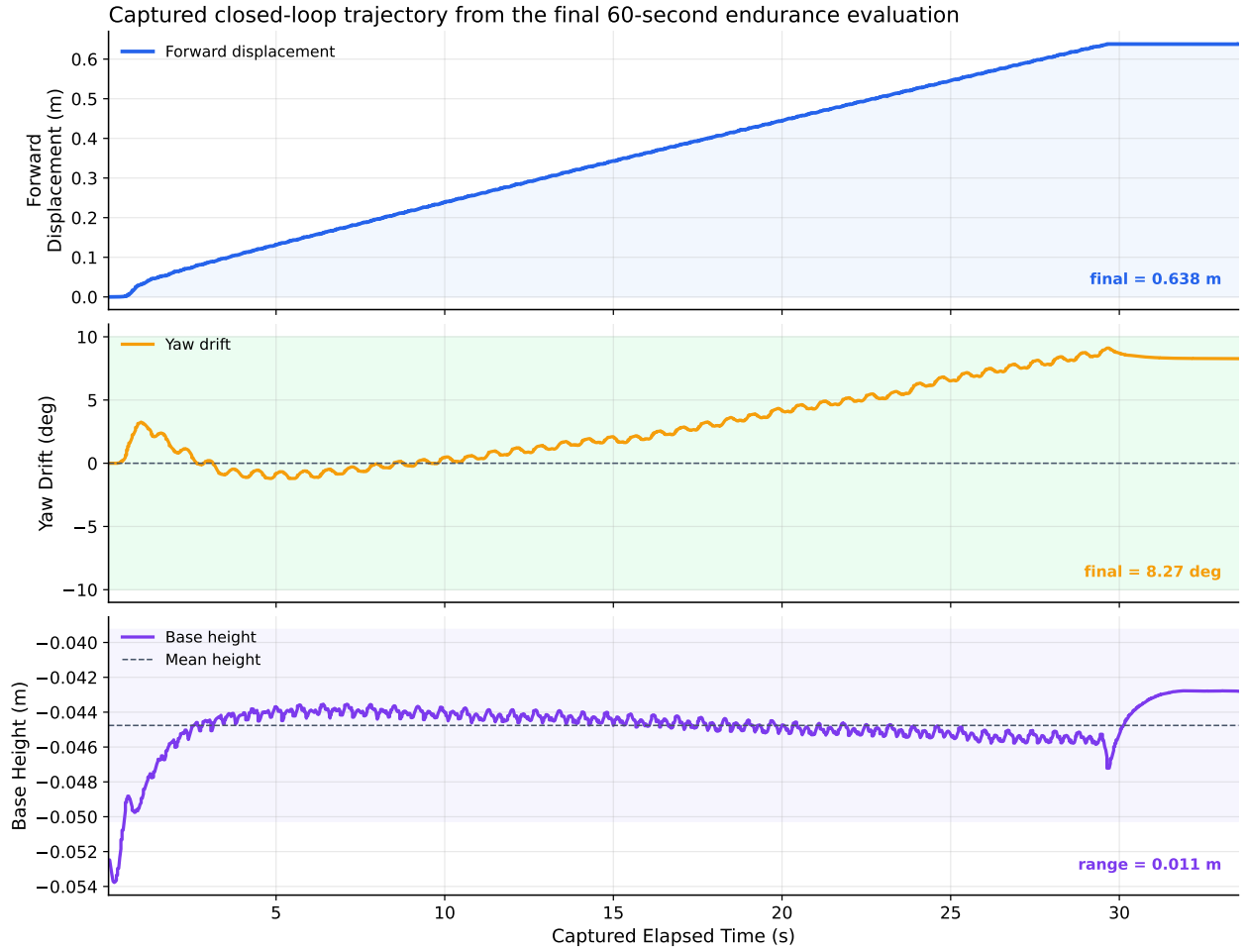


Fig. 5: Time-series extracted from the captured odometry window of the final 60-second endurance evaluation. Top: forward displacement grows monotonically and reaches approximately 0.638 m by the end of the recorded window. Middle: yaw drift remains bounded and stays within a practically controlled range rather than diverging into the large heading excursions observed in unstable long-horizon trials. Bottom: base height varies only at the centimeter scale, with an overall range of about 0.011 m. Together, these traces provide visual evidence that the proposed controller preserved sustained progression, bounded heading error, and stable body posture during long-horizon walking.

explicit hard *branch-consistency constraint*, the controller no longer treats physical branch validity as a post-processing preference. Instead, it encodes that validity directly into the admissible solution set and uses a solver whose constraint handling remains active in the regime that matters for locomotion. This distinction is conceptually important. Runtime guards can prevent damage, but they cannot repair a flawed optimization problem. In contrast, feasible-set shaping addresses the pathology at its source.

We are careful not to overstate the result: the present experiments do not constitute a complete formal proof of safety for arbitrary terrain, disturbances, or contact uncertainty. However, they do establish a much stronger foundation for such guarantees. Once physically impossible or undesired branches are excluded at the solver level, future safety arguments can be built around a controller whose optimization domain is already aligned with mechanism-level reality. In that sense, the

proposed formulation points toward a more principled pathway to *provable safety*: not by adding increasingly aggressive post hoc rejection logic, but by ensuring that unsafe branches are structurally absent from the optimization problem itself.

This perspective is likely to become even more important for locomotion over complex terrain, where the dimensionality of the contact-planning problem and the severity of nonlinear coupling both increase. As legged robots move toward more agile behaviors and more demanding environments, solver accuracy can no longer be evaluated purely in terms of objective convergence or nominal speed. It must also be judged by whether the solver preserves the physical semantics of the mechanism. The results presented here suggest that interior-point methods, when paired with physically meaningful hard constraints, provide a promising and scalable direction for future multi-legged robot MPC.

V. LIMITATIONS AND FUTURE WORK

While the reported results establish a clear solver-level improvement, several limitations should be stated explicitly. First, all endurance evaluations in this paper were conducted on flat terrain in Isaac Sim. This setting is appropriate for isolating the branch-consistency failure mode and for validating the effect of the IPM-based repair under controlled conditions, but it does not exhaust the range of contact phenomena encountered by field robots. In particular, rough terrain and deformable terrain introduce larger contact uncertainty, changing support geometry, and stronger model mismatch, all of which can interact with the optimizer in ways that are not fully exercised by flat-ground experiments. A necessary next step is therefore to extend the proposed IPM + branch-constraint framework to perceptive locomotion over uneven and compliant surfaces, and to evaluate whether the same feasible-set shaping strategy remains robust when contact feedback becomes substantially less predictable.

Second, although the proposed controller preserved stable long-horizon walking, the forward distance did not scale perfectly linearly with time in the 60-second endurance evaluation. The final displacement of 0.638 m confirms persistent forward motion and real-time stability, yet it also indicates that the system gradually adopts a more conservative operating regime over very long horizons. This behavior likely reflects the interaction between heuristic footstep planning, moving-goal reference generation, and small but cumulative state-estimation or target-reconstruction errors. In other words, once the catastrophic branch-switching pathology is removed, a more subtle limitation emerges: the controller remains safe and stable, but its upper-layer motion-generation heuristics may still bias it toward long-horizon stabilization rather than sustained aggressive progression. Addressing this non-linear distance scaling will require improvements in the trajectory generator, foothold policy, and heading-regulation logic, rather than further changes to the branch-consistency constraint itself.

These limitations also help position the broader significance of the paper. Our contribution is strongest at the level of solver correctness and mechanism-aware feasibility: we show that branch validity must be treated as an optimization-domain property rather than as a runtime patch. Future work should build on this corrected solver foundation in two directions. One direction is environmental complexity, including rough, deformable, and partially observed terrain. The other is long-horizon performance shaping, including better footstep heuristics, improved reference generation, and tighter coupling between perception, locomotion intent, and whole-body dynamics. Together, these extensions would test whether the branch-safe IPM formulation introduced here can serve not only as a fix for a flat-ground failure mode, but as a scalable design principle for high-dimensional legged locomotion in realistic settings.

VI. CONCLUSION

This paper addressed a subtle but high-impact failure mode in hexapod MPC: the optimizer could converge to physically

inconsistent tarsus branches even while producing task-space-plausible trajectories. We showed that the problem was not primarily caused by gait timing, weight tuning, or runtime guard logic, but by insufficient solver-level enforcement of branch admissibility in the original SQP pipeline. To resolve this issue, we introduced a hard *TarsusBranchConstraint* and solved the resulting optimal control problem with an Interior-Point Method.

The resulting controller eliminates branch-guard interventions in the reported experiments, restores stable forward progression, improves tracking accuracy, and remains comfortably within real-time computational limits. In addition to the short-horizon A/B comparison against the SQP baseline, the controller also sustained stable medium- and long-horizon endurance walking while preserving bounded yaw drift, limited base-height variation, and sub-5-ms average solve time. Beyond the immediate performance gain, the more important outcome is architectural: physical branch validity is now enforced inside the optimization problem rather than repaired after optimization. This shifts the controller from reactive damage control to proactive feasible-set design, which is essential for dependable legged locomotion.

The broader implication is that high-dimensional multi-legged systems require solver formulations that preserve mechanism semantics, not merely objective convergence. In such systems, deceptive local minima can appear numerically benign while remaining physically invalid. Our results suggest that IPM combined with explicit mechanism-aware hard constraints offers a promising direction for avoiding these pitfalls and for building MPC controllers with stronger safety interpretations.

Taken together, the paper shows that solver-visible branch-consistency constraints are not merely a numerical convenience, but a prerequisite for reliable online hexapod MPC. By pairing these constraints with an IPM backend and by documenting the engineering conditions required for reproducible real-time performance, we provide a controller design that is both scientifically interpretable and practically deployable.

REFERENCES

- [1] J. D. Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT cheetah 3 through convex model-predictive control," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8594448>
- [2] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive locomotion through nonlinear model predictive control," *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3402–3421, 2023. [Online]. Available: <https://arxiv.org/abs/2208.08373>
- [3] A. Meduri, P. Shah, J. Viereck, M. Khadiv, I. Havoutis, and L. Righetti, "BiConMP: A nonlinear model predictive control framework for whole body motion planning," 2022. [Online]. Available: <https://arxiv.org/abs/2201.07601>
- [4] J.-P. Sleiman, F. Farshidian, and M. Hutter, "Versatile multicontact planning and control for legged loco-manipulation," *Science Robotics*, vol. 8, no. 81, p. eadg5014, 2023. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/37585544/>
- [5] M. Posa, C. Cantu, and R. Tedrake, "A direct method for trajectory optimization of rigid bodies through contact," *The International Journal of Robotics Research*, vol. 33, no. 1, 2014. [Online]. Available: https://groups.csail.mit.edu/robotics-center/public_papers/Posa13.pdf

- [6] C. Mastalli, W. Merkt, G. Xin, J. Shim, M. Mistry, I. Havoutis, and S. Vijayakumar, "Agile maneuvers in legged robots: A predictive control approach," 2022. [Online]. Available: <https://arxiv.org/abs/2203.07554>
- [7] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006. [Online]. Available: <https://optimization-online.org/2004/03/836/>
- [8] G. Frison and M. Diehl, "HPIPM: A high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020. [Online]. Available: <https://arxiv.org/abs/2003.02547>